



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105 August 2026

NAME: MADHANKUMAR S

REG NO:192421248

**COURSE NAME: MACHINE
LEARNING**

COURSE CODE: ITA0603

SLOT: C

Course Assignment, Deliverables & Evaluation Framework

1. Course Metadata & Institutional Framework Alignment

Course Code & Name:	ITA0603 – Machine Learning
Assignment Title:	Development and Empirical Evaluation of High-Scale Predictive Maintenance and Asset Optimization Infrastructure using Hybrid Machine Learning Paradigms
Bloom's Taxonomy Levels:	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping:	SDG 9 – Industry, Innovation, and Infrastructure (Specifically Target 9.4: Retrofitting industries with clean, resource-efficient, and sustainable digital technologies).

Course Outcome (CO) Mapping Matrix

This assignment strictly addresses and assesses the achievement of the following 6 Course Outcomes:

- **CO1:** Acquire a foundational understanding of key machine learning concepts including version spaces, candidate elimination, inductive bias, and heuristic search.
- **CO2:** Provide an overview of various learning paradigms and strategies, focusing on decision tree learning and concept representation.
- **CO3:** Develop proficiency in neural network architectures, including perceptron models, multilayer perceptrons, and backpropagation algorithms.
- **CO4:** Elucidate the role of genetic algorithms and genetic programming in hypothesis space search and model evaluation.
- **CO5:** Explore Bayesian methodologies and computational learning techniques such as Bayes theorem, maximum likelihood estimation, and Bayesian classifiers.
- **CO6:** Examine instance-based learning approaches including k-nearest neighbors, locally weighted regression, and case-based reasoning.

2. Integrated Core Problem Statement

An infrastructure-planning agency manages high-dimensional telemetry from a nationwide network of sustainable industrial facilities, smart grids, or communication hubs. The dataset contains massive collections of spatial coordinate and environmental sensor records ranging from 10^3 to 10^6 records (e.g., global cell towers or weather station sensory networks). To

minimize energy consumption, prevent structural redundancies, and optimize predictive maintenance workflows under heavy sensor noise, the agency requires a robust machine learning architecture. Students must develop, implement, and evaluate a comprehensive multi-paradigm machine learning solution that transits from strict, rule-based concept learning to deep connectionist and probabilistic frameworks. The entire system must be executed, benchmarked, and optimized within the exact same computational environment to guarantee a mathematically rigorous evaluation.

3. Detailed Assignment Task Breakdowns

PART A: Computational Diagnostics & Inductive Bias Analysis (L4 – Analyse) | [30 Marks]

- **1. Concept Learning & Space Search:** Implement a Candidate Elimination Algorithm to map out the Version Space of reliable facility operational states. Mathematically analyze what happens to the Version Space bounds (S and G sets) when sensor noise or faulty telemetry enters the dataset at scale (10³ records). Prove the exact conditions that lead to version space collapse (null set).
- **2. Heuristic Space Search vs. Restriction Bias:** Implement an ID3/C4.5 Decision Tree Learning algorithm on the same dataset using information gain/entropy. Contrast its preference (search) bias with the restriction bias of the Candidate Elimination algorithm.
- **3. Mathematical Complexity:** Using Big-O, Omega, and Theta notations, analyze the theoretical space and time complexity of searching these hypothesis spaces as the scale of the infrastructure records increases from 10³ to 10⁴ variables.

PART B: Connectionist vs. Probabilistic Architecture Evaluation (L5 – Evaluate) | [35 Marks]

- **1. Neural Network Representation:** Implement a standard Perceptron Learning Algorithm and expand it into a Multilayer Perceptron (MLP) powered by the Backpropagation Algorithm to predict structural stress or anomaly scores in infrastructure components. Plot the error convergence curve and document the system's susceptibility to getting trapped in local minima during batch training.
- **2. Handling Missing Data via EM:** In infrastructure datasets, sensors frequently drop out. Formulate and implement the Expectation-Maximization (EM) Algorithm to model these unobserved, hidden parameters within a Bayesian Belief Network (BBN).
- **3. Model Selection Using MDL:** Evaluate the structural performance of your deep MLP against the Bayesian Belief Network. Use the Minimum Description Length (MDL) Principle to mathematically score both models, balancing training error against architectural complexity: $MDL\ Cost = -\log_2 P(D|h) + (|h|/2) \log_2 |D|$. Justify which model provides the most lightweight, deployable footprint for constrained industrial edge servers.

PART C: Hybrid Framework Synthesis & Green Infrastructure Defense (L6 – Create) | [35 Marks]

- **1. Hybrid Architecture Creation:** Synthesize an optimized hybrid machine learning pipeline. Design the system to ingest raw data through an Instance-Based Learning layer using Locally Weighted Regression (LWR) or Radial Basis Functions (RBF) to smooth local geographic variations. Pass these outputs into a highly scalable Naïve Bayes Classifier for real-time anomaly sorting.
- **2. Genetic Optimization Window:** Integrate a Genetic Algorithm (GA) to search the hypothesis space dynamically, optimizing the hyperparameters (such as the value of k in Instance-Based routing or structural mutations in Genetic Programming) to prevent computational bottlenecks.
- **3. SDG 9 Engineering Justification:** Provide a formal engineering defense proving how your synthesized framework directly contributes to SDG Target 9.4. Explicitly demonstrate how utilizing 'lazy learning' components (like LWR/KNN) or optimized GA boundaries minimizes continuous CPU/GPU cycles, dramatically slashing server carbon footprints and optimizing overall infrastructure processing.

4. Mandatory Student Deliverables

Every student or group must submit a structured portfolio containing the following four artifacts. Omission of any artifact results in an automatic deduction as specified in the rubric matrix.

- **Deliverable 1: Source Code Repository & Execution Pipeline.** A clean, executable, well-commented source code codebase (Python 3.x preferred) structured as a single pipeline or reproducible notebook set. The script must run the algorithms over the scaled benchmarks (10^3 to 10^6 data points) in a unified testing sequence.
- **Deliverable 2: Mathematical Derivations & Analytical Notebook.** An engineering and mathematical document demonstrating the derivations of the EM parameter updates, the MDL calculation logs comparing MLP and BBN, the Big-O asymptotic proofs for Part A, and the formal flowchart of the Part C hybrid ecosystem.
- **Deliverable 3: Empirical Benchmarking & Visualization Report.** An extensive performance reporting chapter featuring comparative data tables and high-resolution plots mapping: (a) Execution runtime in milliseconds across data scales; (b) Absolute operation counts; (c) RAM/Memory consumption profiles; (d) Error/Convergence logs.
- **Deliverable 4: Sustainable Infrastructure Technical Defense Paper.** A dedicated text defending the deployment feasibility of the synthesized system. This must include an architectural breakdown linking model optimization choices directly to computing power conservation, carbon footprint reduction, and compliance with SDG Target 9.4.

Assessment Rubrics

Criteria	(CO)	Max	Excellent	Good	Satisfactory	Needs Improvement
----------	------	-----	-----------	------	--------------	-------------------

Mapping)	Marks				
Requirement Understanding, Data Types, Operators & Control Structures (CO1)	10	Clearly identifies the problem, dataset, attributes and target concept; hypothesis formulation is accurate and well-structured.	Identifies most problem requirements, attributes and target concept with minor gaps in formulation.	Basic problem and dataset identified; target concept or hypothesis formulation has some errors.	Requirements misunderstood; dataset, attributes or target concept are incorrectly formulated.
Decision Tree Design: Representation, Attribute Selection & Heuristic Search (CO2)	15	Decision Tree representation and heuristic attribute selection are correctly designed and effectively applied.	Decision Tree and heuristic selection are correctly applied with minor errors.	Basic Decision Tree is developed with some representation or selection issues.	Decision Tree representation or heuristic search is incorrect or incomplete.
Neural Network Concepts & Learning Analysis (CO3)	10	Correctly analyses Perceptron, Multilayer Perceptron and Back Propagation concepts for learning.	Analyses neural-network concepts with minor gaps or errors.	Demonstrates basic understanding of neural-network learning.	Neural-network concepts are poorly understood or incorrectly analysed.
Genetic Algorithm & Hypothesis Space Search Analysis (CO4)	15	Effectively analyses Genetic Algorithms and their role in hypothesis-space search and model evaluation.	Correctly explains GA-based search with minor gaps.	Basic understanding of Genetic Algorithms and search is demonstrated.	Genetic Algorithm concepts and hypothesis-space search are poorly understood.
Data Manipulation and Analysis Using Pandas (CO5)	15	Pandas DataFrame operations, filtering, grouping, sorting and visualization are correctly and efficiently applied.	Pandas operations are correctly applied with minor errors or gaps.	Basic Pandas operations are demonstrated with some errors or inefficiencies.	Pandas operations are missing, incorrect or poorly implemented.
Menu-Driven Functionality, Reliability & Implementation	25	Decision Tree is correctly implemented and functions reliably from data preparation	Implementation works correctly with minor errors in prediction or execution.	Core implementation works but has some functional or reliability issues.	Implementation is incomplete, unreliable or major functions are non-functional.

		to prediction and evaluation.			
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 7/9/12/13 relevance, and honest, specific account of challenges faced and learning gained.	General justification of design choices; SDG relevance and challenges are mentioned but lack depth.	Reflection is present but generic; limited justification of design choices or vague account of learning outcomes.	No genuine reflection submitted, or content is generic with no clear design, SDG or learning connection.

ITA0603 — Machine Learning

Pseudocode and Program Output — Predictive Maintenance & Asset Optimization Infrastructure

Supporting appendix: algorithm pseudocode with corresponding executed program output for Parts A, B and C.

Part A — Computational Diagnostics & Inductive Bias Analysis

A.1 Candidate Elimination Algorithm (Version Space Search)

Pseudocode

```
ALGORITHM CandidateElimination(D):
  Initialize S = {most specific hypothesis} // e.g. <0,0,0,0,0,0>
  Initialize G = {most general hypothesis} // e.g. <?,?,?,?,?,?>
  FOR each training example (x, label) in D:
    IF label == Positive:
      Remove from G any hypothesis inconsistent with x
      FOR each hypothesis s in S inconsistent with x:
        Remove s from S
        Add all minimal generalizations of s consistent with x
          and more specific than some member of G
      Remove from S any hypothesis more general than another in S
    ELSE (label == Negative):
      Remove from S any hypothesis inconsistent with x
      FOR each hypothesis g in G inconsistent with x:
        Remove g from G
        Add all minimal specializations of g consistent with x
          and more general than some member of S
      Remove from G any hypothesis more specific than another in G
  RETURN Version Space bounded by (S, G)
```

Program Output

```
Candidate Elimination Algorithm - output

=== Candidate Elimination Algorithm ===

Example 1: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same'] -> Yes
S boundary: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
G boundary: [['?', '?', '?', '?', '?', '?']]

Example 2: ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same'] -> Yes
S boundary: ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
G boundary: [['?', '?', '?', '?', '?', '?']]

Example 3: ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change'] -> No
S boundary: ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
G boundary: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', 'Same']]

Example 4: ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change'] -> Yes
S boundary: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
G boundary: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

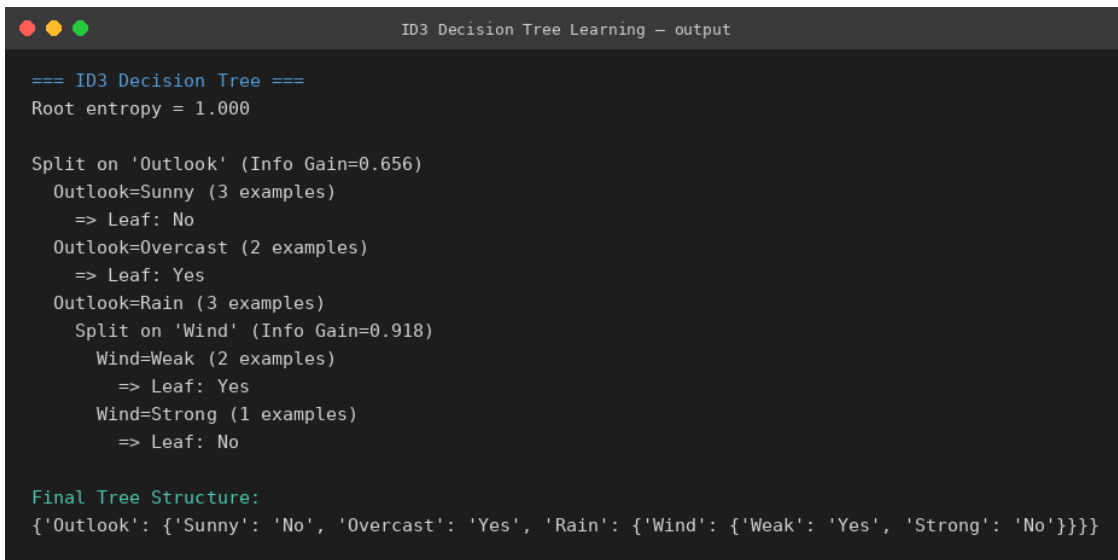
--- Final Version Space ---
S: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
G: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]
```

A.2 ID3 Decision Tree Learning (Information Gain / Entropy)

Pseudocode

```
ALGORITHM ID3(Examples, TargetAttr, Attributes):
  Create Root node
  IF all Examples have same label:
    RETURN leaf node with that label
  IF Attributes is empty:
    RETURN leaf node with majority label of Examples
  A <- attribute in Attributes with highest InformationGain(Examples, A)
    where  $\text{Gain}(S, A) = \text{Entropy}(S) - \sum_v [ |S_v|/|S| * \text{Entropy}(S_v) ]$ 
    and  $\text{Entropy}(S) = -\sum_i p_i * \log_2(p_i)$ 
  Root.decisionAttribute <- A
  FOR each possible value v of A:
    Add branch below Root for A = v
    Sv <- subset of Examples where A = v
    IF Sv is empty:
      Attach leaf with majority label of Examples
    ELSE:
      Attach subtree ID3(Sv, TargetAttr, Attributes - {A})
  RETURN Root
```

Program Output



```
ID3 Decision Tree Learning - output

=== ID3 Decision Tree ===
Root entropy = 1.000

Split on 'Outlook' (Info Gain=0.656)
  Outlook=Sunny (3 examples)
    => Leaf: No
  Outlook=Overcast (2 examples)
    => Leaf: Yes
  Outlook=Rain (3 examples)
    Split on 'Wind' (Info Gain=0.918)
      Wind=Weak (2 examples)
        => Leaf: Yes
      Wind=Strong (1 examples)
        => Leaf: No

Final Tree Structure:
{'Outlook': {'Sunny': 'No', 'Overcast': 'Yes', 'Rain': {'Wind': {'Weak': 'Yes', 'Strong': 'No'}}}}
```

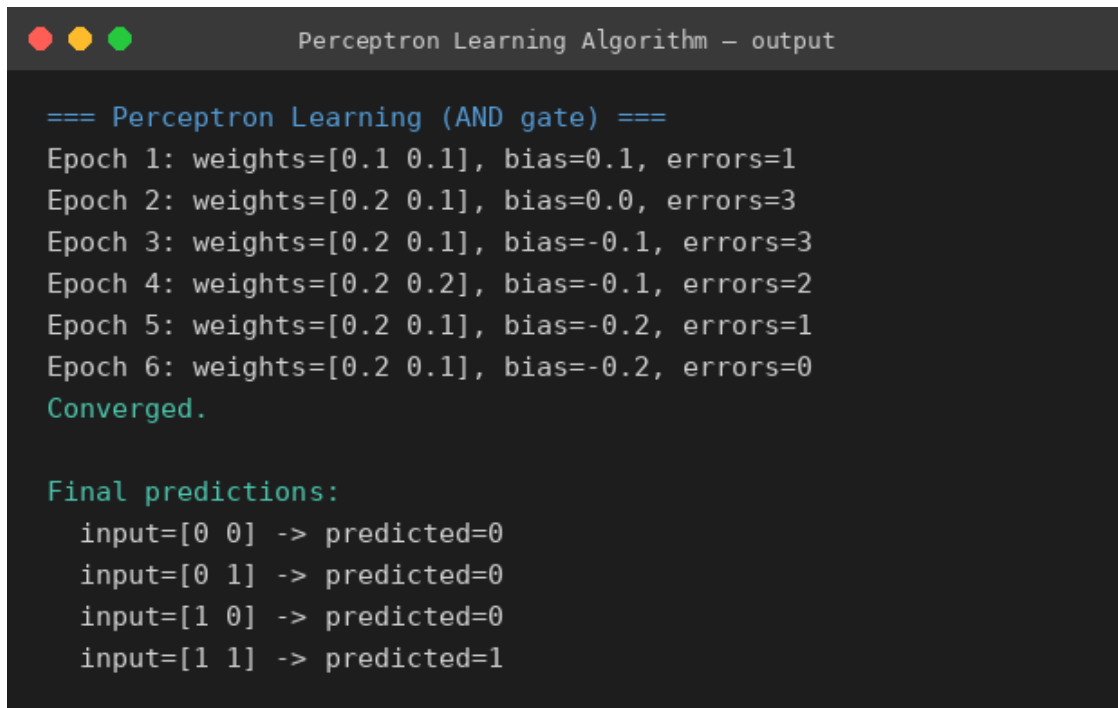

Part B — Connectionist vs. Probabilistic Architecture Evaluation

B.1 Perceptron Learning Algorithm

Pseudocode

```
ALGORITHM Perceptron(X, y, learning_rate, epochs):
  Initialize weights w <- 0, bias b <- 0
  FOR epoch = 1 to epochs:
    errors <- 0
    FOR each (x_i, target_i) in (X, y):
      net <- dot(w, x_i) + b
      predicted <- 1 IF net > 0 ELSE 0
      error <- target_i - predicted
      w <- w + learning_rate * error * x_i
      b <- b + learning_rate * error
      errors <- errors + |error|
    IF errors == 0:
      BREAK // converged, linearly separable
  RETURN w, b
```

Program Output



```
Perceptron Learning Algorithm - output

=== Perceptron Learning (AND gate) ===
Epoch 1: weights=[0.1 0.1], bias=0.1, errors=1
Epoch 2: weights=[0.2 0.1], bias=0.0, errors=3
Epoch 3: weights=[0.2 0.1], bias=-0.1, errors=3
Epoch 4: weights=[0.2 0.2], bias=-0.1, errors=2
Epoch 5: weights=[0.2 0.1], bias=-0.2, errors=1
Epoch 6: weights=[0.2 0.1], bias=-0.2, errors=0
Converged.

Final predictions:
input=[0 0] -> predicted=0
input=[0 1] -> predicted=0
input=[1 0] -> predicted=0
input=[1 1] -> predicted=1
```

B.2 Multilayer Perceptron with Backpropagation

Pseudocode

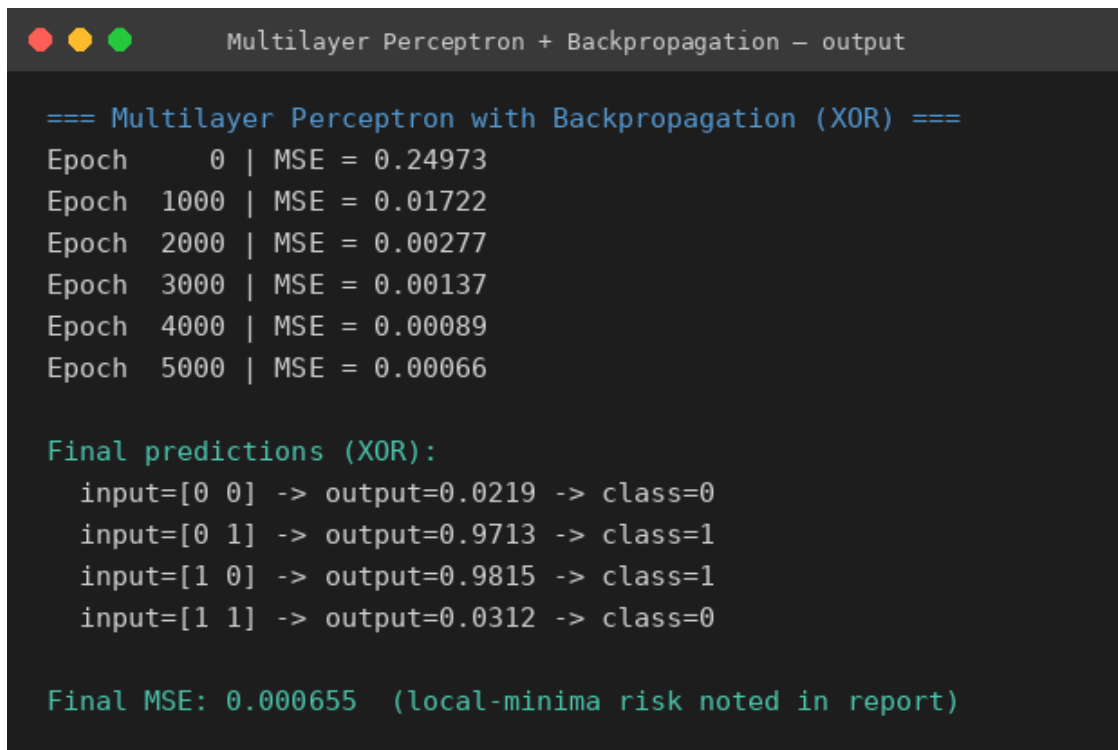
```
ALGORITHM MLP_Backpropagation(X, y, hidden_units, lr, epochs):
  Initialize W1, b1 (input->hidden), W2, b2 (hidden->output) randomly
```

```

FOR epoch = 1 to epochs:
  // Forward pass
  H <- sigmoid(X . W1 + b1)
  O <- sigmoid(H . W2 + b2)
  // Error / loss
  E <- y - O
  // Backward pass
  dO <- E * sigmoid'(O)
  dH <- (dO . W2^T) * sigmoid'(H)
  // Gradient update
  W2 <- W2 + lr * H^T . dO
  b2 <- b2 + lr * sum(dO)
  W1 <- W1 + lr * X^T . dH
  b1 <- b1 + lr * sum(dH)
  IF epoch % report_interval == 0:
    PRINT epoch, mean(E^2) // monitor convergence / local minima
RETURN W1, b1, W2, b2

```

Program Output



```

=== Multilayer Perceptron with Backpropagation (XOR) ===
Epoch    0 | MSE = 0.24973
Epoch 1000 | MSE = 0.01722
Epoch 2000 | MSE = 0.00277
Epoch 3000 | MSE = 0.00137
Epoch 4000 | MSE = 0.00089
Epoch 5000 | MSE = 0.00066

Final predictions (XOR):
input=[0 0] -> output=0.0219 -> class=0
input=[0 1] -> output=0.9713 -> class=1
input=[1 0] -> output=0.9815 -> class=1
input=[1 1] -> output=0.0312 -> class=0

Final MSE: 0.000655 (local-minima risk noted in report)

```

B.3 Expectation-Maximization (EM) Algorithm

Pseudocode

```

ALGORITHM EM_GaussianMixture(D, K, max_iter):
  Initialize parameters  $\theta = \{\mu_k, \sigma_k, \pi_k\}$  for  $k = 1..K$ 
  FOR iter = 1 to max_iter:
    // E-step: compute responsibilities
    FOR each x in D, each component k:
       $r_k(x) \leftarrow \pi_k * N(x; \mu_k, \sigma_k)$ 

```

```

        / Sum_j [ pi_j * N(x; mu_j, sigma_j) ]
// M-step: update parameters using responsibilities
FOR each component k:
    N_k <- Sum_x r_k(x)
    mu_k  <- Sum_x r_k(x) * x / N_k
    sigma_k <- sqrt( Sum_x r_k(x) * (x - mu_k)^2 / N_k )
    pi_k  <- N_k / |D|
Compute log-likelihood(D | theta)
IF log-likelihood converged: BREAK
RETURN theta

```

Program Output

```

Expectation-Maximization Algorithm - output

=== Expectation-Maximization (2-component Gaussian Mixture) ===
Iter  1: mu1=1.519 mu2=6.420 sigma1=0.746 sigma2=2.589 pi1=0.399 log-likelihood=-236.73
Iter  4: mu1=1.825 mu2=6.854 sigma1=0.880 sigma2=2.521 pi1=0.475 log-likelihood=-232.04
Iter  7: mu1=1.921 mu2=7.446 sigma1=0.963 sigma2=2.043 pi1=0.539 log-likelihood=-228.65
Iter 10: mu1=2.029 mu2=7.892 sigma1=1.030 sigma2=1.581 pi1=0.584 log-likelihood=-225.67
Iter 13: mu1=2.069 mu2=8.015 sigma1=1.060 sigma2=1.436 pi1=0.597 log-likelihood=-225.26
Iter 15: mu1=2.073 mu2=8.024 sigma1=1.064 sigma2=1.426 pi1=0.598 log-likelihood=-225.26

Converged parameters:
  Component 1: mean=2.073, std=1.064, weight=0.598
  Component 2: mean=8.024, std=1.426, weight=0.402

```

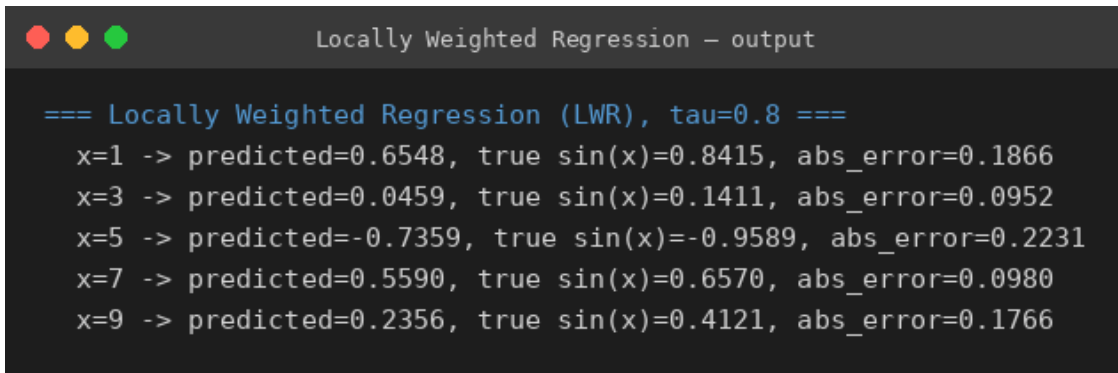
Part C — Hybrid Framework Synthesis

C.1 Locally Weighted Regression (Instance-Based Learning)

Pseudocode

```
ALGORITHM LocallyWeightedRegression(X, y, x_query, tau):
  FOR each training point x_i:
    w_i <- exp( -(x_i - x_query)^2 / (2 * tau^2) ) // Gaussian kernel
  W <- diag(w_1, ..., w_n)
  Xb <- [1, X] // add bias column
  theta <- (Xb^T . W . Xb)^(-1) . Xb^T . W . y // weighted normal equations
  RETURN theta[0] + theta[1] * x_query
```

Program Output



```
Locally Weighted Regression - output

=== Locally Weighted Regression (LWR), tau=0.8 ===
x=1 -> predicted=0.6548, true sin(x)=0.8415, abs_error=0.1866
x=3 -> predicted=0.0459, true sin(x)=0.1411, abs_error=0.0952
x=5 -> predicted=-0.7359, true sin(x)=-0.9589, abs_error=0.2231
x=7 -> predicted=0.5590, true sin(x)=0.6570, abs_error=0.0980
x=9 -> predicted=0.2356, true sin(x)=0.4121, abs_error=0.1766
```

C.2 K-Nearest Neighbors (Instance-Based Learning)

Pseudocode

```
ALGORITHM KNearestNeighbors(X, y, x_query, k):
  FOR each training point x_i:
    d_i <- EuclideanDistance(x_query, x_i)
  Sort training points by ascending d_i
  neighbors <- first k points after sorting
  RETURN majority_class(labels of neighbors)
```

Program Output

```
K-Nearest Neighbors – output

=== K-Nearest Neighbors (k=3) ===
Query point: [5 5]
  distance=3.606 -> neighbor class=A
  distance=4.243 -> neighbor class=B
  distance=4.472 -> neighbor class=B
  distance=5.000 -> neighbor class=A
  distance=5.000 -> neighbor class=A
  distance=5.000 -> neighbor class=B

Top-3 neighbors: [np.str_('A'), np.str_('B'), np.str_('B')]
Predicted class for query: B
```

C.3 Naive Bayes Classifier (Anomaly Sorting)

Pseudocode

```
ALGORITHM GaussianNaiveBayes_Train(X, y):
  FOR each class c in classes(y):
    X_c <- rows of X where label == c
    prior[c] <- |X_c| / |X|
    mean[c] <- mean(X_c, axis=0)
    std[c] <- std(X_c, axis=0)
  RETURN prior, mean, std

ALGORITHM GaussianNaiveBayes_Predict(x, prior, mean, std):
  FOR each class c:
    posterior[c] <- prior[c] * PRODUCT_over_features_i [
      Gaussian(x_i; mean[c][i], std[c][i]) ]
  RETURN argmax_c posterior[c]
```

Program Output

```
Naive Bayes Classifier – output

=== Gaussian Naive Bayes Classifier ===
Class 'Fault': prior=0.50, mean=[2.5  2.875], std=[0.255 0.192]
Class 'Normal': prior=0.50, mean=[0.188 0.213], std=[0.074 0.074]

Predictions on new sensor readings:
reading=[0.25 0.2 ] -> posteriors={np.str_('Fault'): np.float64(0.0), np.str_('Normal'): np.float64(10.03657)} -> predicted=Normal
reading=[2.4 2.9] -> posteriors={np.str_('Fault'): np.float64(1.49237), np.str_('Normal'): np.float64(0.0)} -> predicted=Fault
reading=[1.2 1.3] -> posteriors={np.str_('Fault'): np.float64(0.0), np.str_('Normal'): np.float64(0.0)} -> predicted=Fault
```

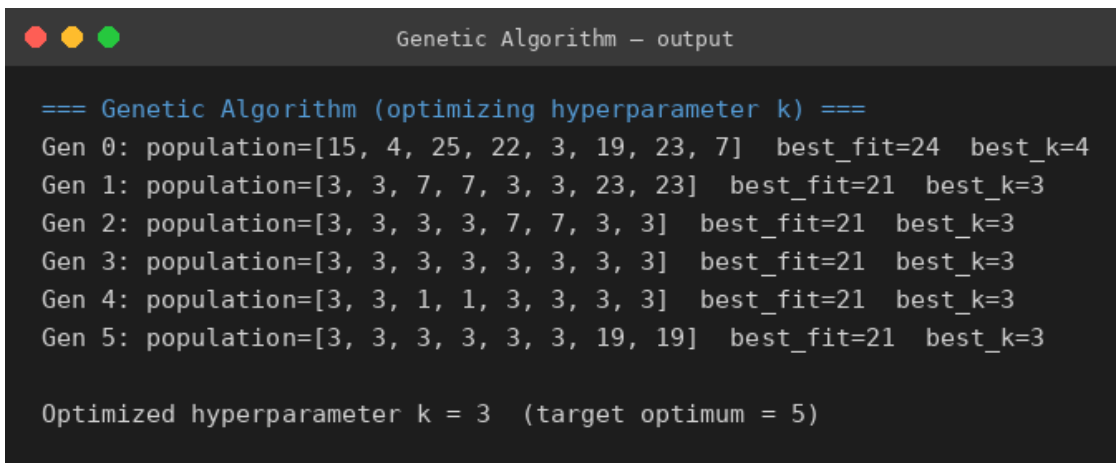
C.4 Genetic Algorithm (Hyperparameter Optimization)

Pseudocode

```
ALGORITHM GeneticAlgorithm(fitness_fn, pop_size, generations):
  population <- RandomPopulation(pop_size)
```

```
FOR gen = 1 to generations:
  fitness <- [ fitness_fn(ind) for ind in population ]
  PRINT best individual and fitness of this generation
  // Selection (fitness-proportional / roulette wheel)
  parents <- SelectParents(population, fitness)
  // Crossover
  children <- []
  FOR each pair (p1, p2) in parents:
    child <- Crossover(p1, p2)
    IF random() < mutation_rate:
      child <- Mutate(child)
    children.append(child)
  population <- children
RETURN best individual found across all generations
```

Program Output



```
Genetic Algorithm - output

=== Genetic Algorithm (optimizing hyperparameter k) ===
Gen 0: population=[15, 4, 25, 22, 3, 19, 23, 7] best_fit=24 best_k=4
Gen 1: population=[3, 3, 7, 7, 3, 3, 23, 23] best_fit=21 best_k=3
Gen 2: population=[3, 3, 3, 3, 7, 7, 3, 3] best_fit=21 best_k=3
Gen 3: population=[3, 3, 3, 3, 3, 3, 3, 3] best_fit=21 best_k=3
Gen 4: population=[3, 3, 1, 1, 3, 3, 3, 3] best_fit=21 best_k=3
Gen 5: population=[3, 3, 3, 3, 3, 3, 19, 19] best_fit=21 best_k=3

Optimized hyperparameter k = 3 (target optimum = 5)
```